Agent Workflow

원격 개발 실험에서 단계별 제작 파이프라인으로

김준호 / Programmer

GPT 리모트로 개발을 시도하며 발견한 문제와
이를 해결하기 위해 바꾼 작업 방식, 구현 및 적용 사례

---

01 원격 개발 실험

집에서 누워서도 개발할 수 있을까?

02 이슈 기반 레시피

대화가 바뀌어도 작업 맥락을 이어가기

03 플랜 · 설계 · 구현

단계별 입력과 판단을 분리하기

2026.10 / ProjectSF · UnrealAgentWorkflow

01 / REMOTE DEVELOPMENT

집에서 누워서도 개발할 수 있을까?

GPT 리모트 기능을 접한 뒤, 개발 PC 앞에 앉지 않고도 작업을 진행할 수 있는지 실험했다. 원격으로 작업을 지시하고 결과를 확인하는 것이 출발점이었다.

작업 지시
원격에서 개발 요청

→

개발 진행
Agent가 작업 수행

→

결과 확인
결과를 보고 다음 요청

실험 중 드러난 문제

작업이 길어지거나 대화가 바뀌면 개발 맥락이 유지되지 않았다. 무엇을 만들기로 했는지, 어디까지 진행했는지, 어떤 판단을 내렸는지를 다시 설명해야 했다.

개선이 필요했던 지점

원격에서 명령을 보내는 기능만으로는 여러 번의 대화에 걸친 개발을 이어가기 어려웠다. 작업의 목표와 현재 상태를 대화 밖에 남기고, 다음 작업이 같은 기준에서 시작하도록 만들 필요가 있었다.

다음 단계의 질문

“대화가 끝나도 개발 맥락을 남기고,
새 대화에서 이어서 작업하려면 무엇을 저장해야 할까?”

구현 근거 개발자 실험 경위 / 이후 Issue·Recipe 실행 계약으로 구체화

02 / ISSUE-BASED RECIPE

개발 맥락을 이슈와 레시피에 남기기

컨텍스트 유지 문제를 해결하기 위해 이슈 기반 레시피를 도입했다. 목표와 요구사항, 진행 상태, 검증 결과를 이슈에 남기고 새 대화에서도 해당 기록을 기준으로 작업을 이어가도록 구성했다.

Issue
목표 · 범위 · 결정 · 진척

→

Recipe
제작 입력 · 보존 · 완료 조건

→

Evidence
실행 결과 · 검증 근거

적용 사례 / NeoSeoul 월드 제작

서울 GIS를 입력으로 지형·수계·도로 기반을 만들고, 롯데 지구의 랜드마크와 주변 공간을 이어가는 작업에 적용했다. 여러 작업에 걸친 제작 범위·원본·보존 조건·검증 결과를 이슈에 연결해 다음 작업의 기준으로 사용했다.



이전 NeoSeoul 프로토타입 · 2026.10.07 저장 검증 화면 · 지형·수계와 제작 구역

월드 전체를 한 번에 다시 생성하는 대신 지역과 도메인별로 작업을 나누고, 이미 만든 지형·도로·수계와 사람의 편집을 보존하는 조건을 명시했다.

구현 근거 WorldProduction.md / AgentDrivenGameProduction.md / Issue·Recipe 계약 · 이전 GIS 프로토타입

02 / WORLD PRODUCTION

월드 제작을 나누고 이어가기

지형을 만든 뒤 수계와 도로를 연결하고 지역 콘텐츠를 더하려면, 다음 대화에서도 좌표·제작 범위·현재 결과를 같은 기준으로 이해해야 했다. 이슈와 레시피는 각 작업의 입력과 보존 조건, 남은 검증을 연결하는 역할을 했다.



#57 롯데 지구 rough · 타워·몰·섬과 주변 지형·도로·수계의 공간 관계

제작 기준
GIS 입력 · 좌표 · 지역 범위

→

작업 분리
지형 · 수계 · 도로 · 콘텐츠

→

진행 근거
저장 · 재조회 · Evidence

단계 간 맥락과 실행 상태 유지

공통 입력과 지역 범위를 읽고 해당 도메인의 작업을 진행한다. World Partition의 로드 단위, 제작 지역, 도메인 소유권, 배치 청크를 구분했다. 도로 배치는 checkpoint와 실제 결과를 대조해 불명확한 호출의 중복 실행을 피했다.

당시 확인한 결과

2026.10.07 저장 검증에는 Landscape Edit Layer 8개·Component 128개, 강 구간 11개·호수 17개가 기록돼 있다. 롯데 지구는 공통 월드 기반 위에 지역 콘텐츠를 배치한 rough 단계이며 최종 아트·성능·패키지 검증과 구분했다.

구현 근거 월드 제작·검증 기록 / #54 도로 배치 / #57 지역 구성 · 2026.10.14 방향 전환 이전 사례

03 / PLAN · DESIGN · IMPLEMENTATION

사이트를 활용해 추론 과정을 나누기

이후 사이트 제공 기능을 발견하면서 작업 흐름을 플랜·설계·구현으로 분리했다. 한 대화에서 모든 문제를 함께 다루는 대신, 각 단계가 필요한 입력과 판단에 집중하도록 구성했다.

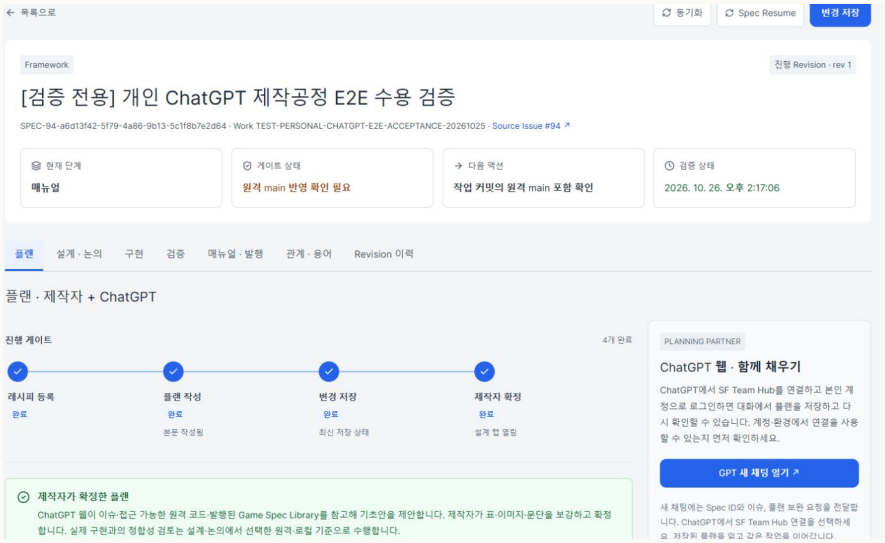
플랜
무엇을 만들 것인가

→

설계
어떻게 만들 것인가

→

구현
확정한 내용을 어떻게 적용할까



현재 공정 UI / SPEC-94 · 요구와 제작 범위를 정리하는 플랜 단계

분리한 입력과 결과

플랜에서는 요구와 범위를 정리하고, 설계에서는 구조·대안·논점을 검토한다. 구현은 확정된 문서와 버전을 입력으로 사용한다. 각 단계의 결과를 저장해 다음 단계에서 같은 기준을 읽도록 했다.

개선 목적

단계별 입력을 제한하고 판단 대상을 명확히 해 추론의 집중도와 일관성을 높이고자 했다. 정량적인 추론 성능 향상률을 측정한 결과로 표현하지 않는다.

구현 근거 SF Team Hub / CreatorWorkflow.md / TeamHubProduction.md

03 / SPEC-75 · WORKFLOW 1/2

플랜을 확정하고 설계 합의를 쌓기

SPEC-75는 어빌리티 시스템 개선 과제를 Spec 워크플로우로 진행한 사례다. 대화에서 요구를 보완하고, 확정한 문서와 합의 기록을 다음 단계의 입력으로 넘겼다.



SPEC-75 실제 설계 공정 UI · 플랜 v4 → 검토·논점 합의 → UML v6 → 구현 인계

01 제작자 + ChatGPT / 플랜 작성·확정

Issue #75의 요구와 기존 원격 코드·문서를 대조하며 목표, 범위, 보존 대상과 완료 조건을 보완했다. 제작자가 플랜을 확정한 뒤 문서 v4를 설계의 공통 입력으로 사용했다.

02 Agent / 확정 플랜의 설계 적합성 검토

저장된 플랜을 읽고 현재 코드·선택한 BP·DT/DA와 대조했다. 재사용·변경·보존·미확인 대상을 나눠 검토 결과를 Spec에 저장했다. 이 단계의 검토 통과를 구현 완료로 처리하지 않았다.

03 제작자 ↔ Agent / 논점별 방안 검토·합의

검토에서 드러난 질문은 현상·원인·근거와 추천·대안으로 정리했다. 제작자는 방안을 선택하고 필요한 논점을 다시 열어 재검토했다. 실제 이력에는 논점 분석 요청 → 방안 생성 → 합의 → 재개방·재합의가 남아 있다.

04 Agent + 제작자 / UML 갱신·전체 설계 합의

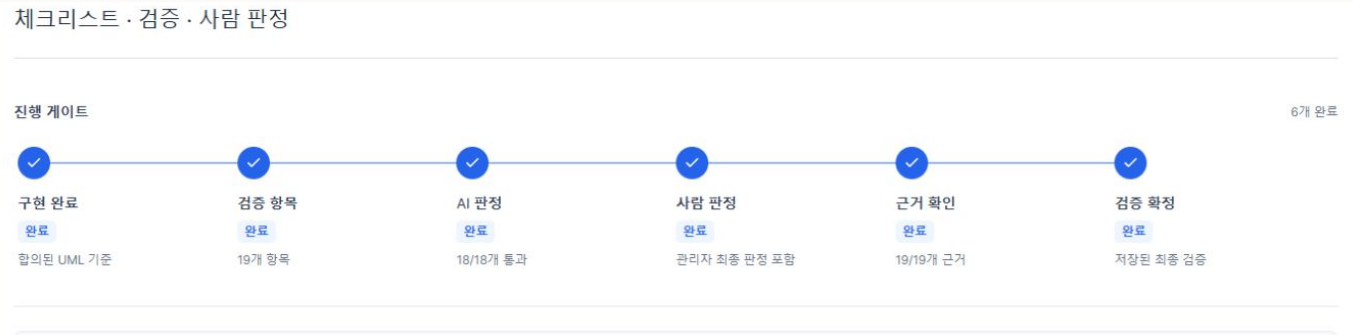
논점의 결론을 반영해 UML을 생성·수정하고 설계를 재검토했다. 개별 논점 합의와 전체 설계 합의를 분리해 기록했으며, 최종 설계 v6을 기준으로 Codex 구현 인계로 이어갔다.

구현 근거 SPEC-75 R3 Store288 / 플랜·설계·공통 이력 280건 대조 / 화면은 현재 보존된 공정 상태

03 / SPEC-75 · WORKFLOW 2/2

구현 인계부터 검증·발행까지 연결

설계 이후에는 구현할 일감과 실제 결과, 사람의 판정, 저장소 반영을 각각 기록했다. 다음 단계는 이전 대화의 요약 대신 Spec에 저장된 문서·버전·근거를 기준으로 진행했다.



SPEC-75 실제 검증 공정 UI · 구현 완료와 AI·사람 판정, 근거 확인, VERIFY를 구분

05 Codex / 구현 적합성 검토·작업 목록

확정 설계를 검토한 뒤 설계 요소와 연결된 구현 항목 18개를 만들었다. 각 항목에 대상, 변경 내용, 선행 작업, 완료 조건과 검증 방법을 담아 실제 코드·에셋 작업으로 인계했다.

06 Codex / 항목별 실행·근거 저장·완료 동기화

작업하면서 실행 결과와 증거를 해당 구현 항목에 기록했다. 항목이 현재 설계와 맞는지, 필요한 근거가 있는지 대조한 뒤 완료 상태를 동기화했다. 대화가 바뀌어도 이 기록을 다시 읽어 진행 상태를 복원할 수 있게 했다.

07 Agent + 제작자 / 검증과 실제 수락

구현 항목에 연결한 체크리스트 19개에서 기대 결과·실제 근거·AI 판정·사람 판정을 따로 남겼다. Agent는 코드·저장 자산·실행 기록을 대조하고 제작자는 실제 플레이를 확인했다. 구현 완료 표시와 검증 확정은 별도로 처리했다.

08 제작자·관리자 / VERIFY·매뉴얼·CURRENT 발행

VERIFY 후 사용·편집 매뉴얼을 정리하고, 원격 main에 작업이 포함됐는지 확인했다. 이 조건을 충족한 뒤 2026.10.18 CURRENT R3를 발행해 문서와 판정 이력을 보존했다.

기록된 결과: 구현 18개 완료 · 검증 19개 사람 PASS · AI 검증 대상 18개 PASS. 실제 플레이 수락은 사람 판정으로 남겼다. 구현 항목·검증 항목·개별 테스트 수는 서로 다른 집계다.

구현 근거 구현·검증 이력 / VERIFY → 매뉴얼 갱신 → REMOTE_MAIN_VERIFIED → CURRENT 발행 기록

03 / TECHNICAL IMPLEMENTATION

단계 간 인계를 서버에서 일관되게 처리

단계 분할을 실제 운영에 사용하려면 누가, 어떤 버전으로, 무슨 결과를 저장했는지 확인해야 했다. 대화와 화면이 달라도 같은 상태를 기준으로 작업하도록 서버 MCP와 저장 계약을 구성했다.

공통 서버 MCP와 계정 귀속

팀원별 플러그인 제작·배포는 채택하지 않았다. 기존 SF Team Hub 서버 MCP를 공통 경로로 사용하고 grant·scope·멤버십과 작성자를 검사한다. 일반 작업과 제작자의 확정·사람 판정 권한을 분리했다.

동시 편집과 재전송

저장은 storeVersion이 같은 경우에만 갱신한다. 이전 버전의 저장은 충돌로 거절한다. 같은 계정의 requestId와 입력 hash를 대조해 동일 요청은 기존 결과를 사용하고, 같은 ID에 다른 입력은 거절한다.

```
request
  -> authenticate + authorize
  -> replay / version / revision validation
  -> persist result + history
  -> receipt + readback
```

장시간 작업의 결과 적용

worker lease와 heartbeat로 실행 소유권을 확인한다. 완료 시 Revision·원본·active job·입력 fingerprint를 다시 대조해 오래된 입력으로 생성된 결과가 최신 상태를 덮어쓰지 않도록 했다.

완료 판정

최신 설계에 연결된 구현 기록과 Evidence를 검사한다. 구현 완료, AI 검증, 사람 수락, VERIFY와 원격 main·CURRENT 발행을 분리해 각 단계의 성공이 다음 단계의 완료로 자동 승계되지 않게 했다.

구현 근거 remote-mcp.ts / page-commands.ts / spec-store.ts / agent-jobs.ts / implementation-sync.ts

04 / RESULTS & REFERENCES

실험에서 발견한 문제를 공정으로 바꾸기

원격 개발 실험

GPT 리모트로 개발을 시도하면서 대화가 바뀔 때 작업 맥락이 유지되지 않는 문제를 확인했다. 원격 조작 가능성에서 작업의 연속성으로 해결 대상을 넓혔다.

이슈 기반 레시피

목표·범위·진행·검증을 대화 밖에 남겼다. 월드 제작 사례에서는 지형·수계·도로·지역 콘텐츠의 입력과 보존 조건, 진행 상태와 검증 근거를 연결해 여러 작업과 대화에 걸쳐 개발 맥락을 유지했다.

플랜·설계·구현 파이프라인

사이트를 활용해 단계별 입력과 판단을 분리했다. SPEC-75에서 제작자와 ChatGPT의 플랜 확정, Agent의 검토와 논점별 합의, UML 갱신, Codex 구현 인계, 근거 기반 검증과 사람 수락, 원격 반영·CURRENT 발행을 실제 기록으로 연결했다.

현재 확인 범위

이후 SPEC-94 승인 범위 6개 항목과 정식 VERIFY, 공통 서버 MCP 저장·재조회 및 새 대화의 상태 복원을 확인했다. Android는 두 계정의 사용자 확인을 기록했고 iOS는 보류했다. SPEC-94의 원격 main 확인·CURRENT 발행은 별도 대기 상태다.

공개 저장소와 구현·검증 현황 ↗

이슈·레시피 방식의 월드 제작 사례 ↗

SPEC-75를 Spec 워크플로우로 진행한 과정 ↗

사이트 전체 공정과 실제 화면 7개 ↗

자료 기준 2026.10.26 · 원격 실험의 경위는 개발자 설명, 적용 사례의 구현과 결과는 ProjectSF 기록에 근거한다. 월드 화면과 수치는 이전 GIS 프로토타입의 당시 기록이며, 완성된 게임 월드의 품질·성능을 의미하지 않는다.